



# [ ] KOM120C PEMROGRAMAN

**K13 | Pekan Ke-14**  
**OOP in Scala**

---

Tim Pengajar Pemrograman IPB University

# Learning Outcome

- Mahasiswa memahami object dalam Scala.
- Mahasiswa memahami paradigma hybrid dalam pemrograman.
- Mahasiswa mampu membuat object dan class serta beberapa konsep OOP dalam Scala.



# OOP in Scala

---

- Scala sebagai sebuah bahasa pemrograman multi-paradigm juga mendukung Object-Oriented Programming (OOP).
- Hampir semua konsep dan fitur OOP yang lazim digunakan, didukung oleh Scala : class dan object, constructor dan instantiasi, encapsulation, inheritance, polymorphism.
- Fitur-fitur OOP yang disediakan oleh Scala, jika diperlukan, dapat dikombinasikan dengan berbagai konsep functional programming untuk menghasilkan paradigma pemrograman yang fleksibel dan powerful.

# Class dan Object dalam Scala

- Class dan object dalam Scala tidak jauh berbeda dengan Java, namun implementasinya agak berbeda.
- **Class** adalah sebuah **blueprint** atau prototype untuk membuat objek. Class ini berisi attributes dan methods yang mendefinisikan properti dan perilaku dari objek yang akan diciptakan.

```
class Orang {  
  var nama: String = "Siapa Saja"  
  var usia: Int = 18  
  
  def display() : Unit = {  
    println("Nama: " + nama);  
    println("Usia: " + usia + " tahun.");  
  }  
}  
  
object Main {  
  def main(args: Array[String]) : Unit = {  
    var obj = new Orang();  
    obj.display();  
  }  
}
```



Nama: Siapa Saja  
Usia: 18 tahun.

# Object dalam Scala

- **Object** dalam Scala adalah sebuah "singleton instance" dari suatu class.
- Object digunakan untuk mendefinisikan anggota static (attributes dan methods) yang terhubung ke class.

```
object Aritmatika {  
  def tambah(x: Int, y: Int): Int = x + y  
  def kurang(x: Int, y: Int): Int = x - y  
  def kali(x: Int, y: Int): Int = x * y  
  def bagi(x: Int, y: Int): Int = x / y  
  def modulo(x: Int, y: Int): Int = x % y  
}  
  
object Main {  
  def main(args: Array[String]) : Unit = {  
    println(Aritmatika.tambah(5,3))  
    println(Aritmatika.kurang(5,3))  
    println(Aritmatika.kali(5,3))  
  }  
}
```



8  
2  
15

# Constructor dalam Scala

- Dalam Scala, constructor didefinisikan di dalam class.
- Dua jenis constructor dalam Scala: Primary dan Auxiliary. **Primary constructor** didefinisikan langsung dengan cara menambahkan parameters setelah nama class seperti contoh berikut ini:

```
class Orang(nama: String, usia: Int)
{
  def display() : Unit =
  {
    println("Nama: " + nama);
    println("Usia: " + usia + " tahun.");
  }
}

object Main
{
  def main(args: Array[String]) : Unit =
  {
    var obj = new Orang("Siapa Saja", 18);
    obj.display();
  }
}
```

# Constructor dalam Scala

- **Auxiliary constructor** didefinisikan menggunakan kata kunci "**def this**".
- Umumnya digunakan untuk memberi nilai default dari atribut class.

```
class Orang(nama: String, usia: Int)
{
  def this(nama: String) = this(nama, 0) // default usia = 0
  def display() : Unit =
  {
    println("Nama: " + nama);
    println("Usia: " + usia + " tahun.");
  }
}
```

```
object Main
{
  def main(args: Array[String]) : Unit =
  {
    var obj = new Orang("Siapa Saja");
    obj.display();
  }
}
```



Nama: Siapa Saja  
Usia: 0 tahun.

# Case Class dalam Scala

- **Case Class** sama dengan class biasa, yang memiliki fitur untuk pemodelan data yang tidak dapat diubah (unchangeable data).
- Kelas ini lebih banyak digunakan dalam pencocokan pola.

```
case class Orang(nama: String, usia: Int)

object HelloWorld {
  def main(args: Array[String]): Unit = {
    // create instance
    val p1 = Orang("Ali", 25)
    val p2 = Orang("Budi", 30)

    // access attributes
    println(p1.nama + " | " + p1.usia)

    // copy
    val p3 = p1.copy(usia=26)
    println(p3)

    // pattern matching
    p2 match {
      case Orang(nama, usia) => println(s>Nama: $nama | Usia: $usia")
    }
  }
}
```



```
Ali | 25
Orang(Ali, 26)
Nama: Budi | Usia: 30
```

# Companion Object

- **Companion Object** (atau "objek teman") adalah object yang memiliki nama yang sama dengan class, dan didefinisikan dalam satu file.
- Mengizinkan mengakses anggota yang private.

```
class Lingkaran(val radius: Double) {  
    import Lingkaran._ // import semua anggota Lingkaran  
    def luas: Double = PI * radius * radius  
}  
  
object Lingkaran {  
    private val PI = 3.14  
}  
  
object Main  
{  
    def main(args: Array[String]) : Unit =  
    {  
        var obj = new Lingkaran(5);  
        println(obj.luas)  
    }  
}
```



78.5

# Inheritance in Scala

- Inheritance pada Scala dilakukan dengan kata kunci extends.
- Method **override** ditunjukkan dengan **kata kunci override**.

```
class Orang(nama: String, usia: Int) {  
  def display() : Unit = {  
    println("Nama: " + nama);  
    println("Usia: " + usia + " tahun.");  
  }  
}  
  
class Mhs(nama: String, usia: Int, ipk: Double) extends Orang(nama, usia) {  
  def displayIPK : Unit = {  
    println("IPK: " + ipk);  
  }  
}  
  
object Main {  
  def main(args: Array[String]) : Unit = {  
    var obj = new Mhs("Jeong Sewoon", 16, 3.78);  
    obj.display  
    obj.displayIPK  
  }  
}
```



Nama: Jeong Sewoon  
Usia: 16 tahun.  
IPK: 3.78

# Abstract Class in Scala

Scala juga mengenal **abstract class** yang mendefinisikan kumpulan atribut dan metode yang bersifat abstract, dan hanya dibuat kelas turunannya, namun tidak dapat dibuat instantiasinya.

```
abstract class Ruang2D {  
  def luas: Double  
}  
  
class Segiempat(panjang: Double, lebar: Double) extends Ruang2D {  
  def this() = this(0.0, 0.0)  
  override def luas: Double = panjang * lebar  
}  
  
class Lingkaran(radius: Double) extends Ruang2D {  
  def this() = this(0.0)  
  override def luas: Double = 3.14 * radius * radius  
}  
  
object Main {  
  def main(args: Array[String]) : Unit = {  
    var objS = new Segiempat(3, 5)  
    var objL = new Lingkaran(5)  
    println(objS.luas)  
    println(objL.luas)  
  }  
}
```



15.0  
78.5

# Polymorphism in Scala

Seperti halnya pada bahasa pemrograman OOP lainnya, Scala juga mendukung fitur **polymorphism**.

```
abstract class Ruang2D {  
  def luas: Double  
}  
  
class Segiempat(panjang: Double, lebar: Double) extends Ruang2D {  
  def this() = this(0.0, 0.0)  
  override def luas: Double = panjang * lebar  
}  
  
class Lingkaran(radius: Double) extends Ruang2D {  
  def this() = this(0.0)  
  override def luas: Double = 3.14 * radius * radius  
}  
  
object Main {  
  def main(args: Array[String]) : Unit = {  
    val daftar = List[Ruang2D](new Segiempat(3,5), new Lingkaran(5))  
    val res = daftar.map(x => x.luas).reduce(_+_)  
    println(res)  
  }  
}
```

➡ 93.5

# Diskusi Kelas

Tulis program Scala yang membuat class BankAccount dengan properti accNumber dan balance (saldo). Implementasikan methods untuk menabung (nilainya positif) atau mengambil uang (nilainya negatif) dari akun tertentu. Input berupa daftar transaksi yang berisi pasangan akun dengan list nilai transaksi (positif berarti menabung, negatif berarti mengambil). Output berupa daftar saldo dari setiap akun yang melakukan transaksi.

Contoh:

```
val transaksi = Map(  
  "ID-1234" -> List(1000, 3000, -500),  
  "ID-8121" -> List(5000)  
)  
accBalance(transaksi)
```



```
ID-1234 : 3500.0  
ID-8121 : 5000.0
```